



PLANNING

PLANNING

Planning is essential to perform any job.

The task of coming up with **a sequence of actions that will achieve a goal** is called planning.

Planning problems can be represented as <states, actions and goals>.

For any planning system, we need **domain description, action specification, and goal description**.

For *classical planning*, we will consider only environments that are fully observable, deterministic, finite, static and discrete. These are called classical planning environments.

In non-classical planning systems, we have partially observable or stochastic environments.

PLANNING

Representation of states:- Planners decompose the world **into logical conditions** and represent a state as a conjunction of **positive literals**.

- In first-order state descriptions, literals must be ground and function-free. E.g. `at(John, Sydney)` allowed. `at(x, y)` or `at(father(John), Sydney)` are not allowed.
- Closed world assumptions. Any condition not mentioned are false.

Representation of goals:- A goal is a partially specified state, represented as a conjunction of positive literals. E.g. `at(John, Delhi)`.

- A state `s` satisfies goal `g`, if it(s) contains all the atoms in `g`.

PLANNING

Representation of actions:- An action is specified in terms of **the preconditions** that must hold before it can be executed and **the effects** that ensure when it is executed.

Ex. An action of flying plane from one airport to another.

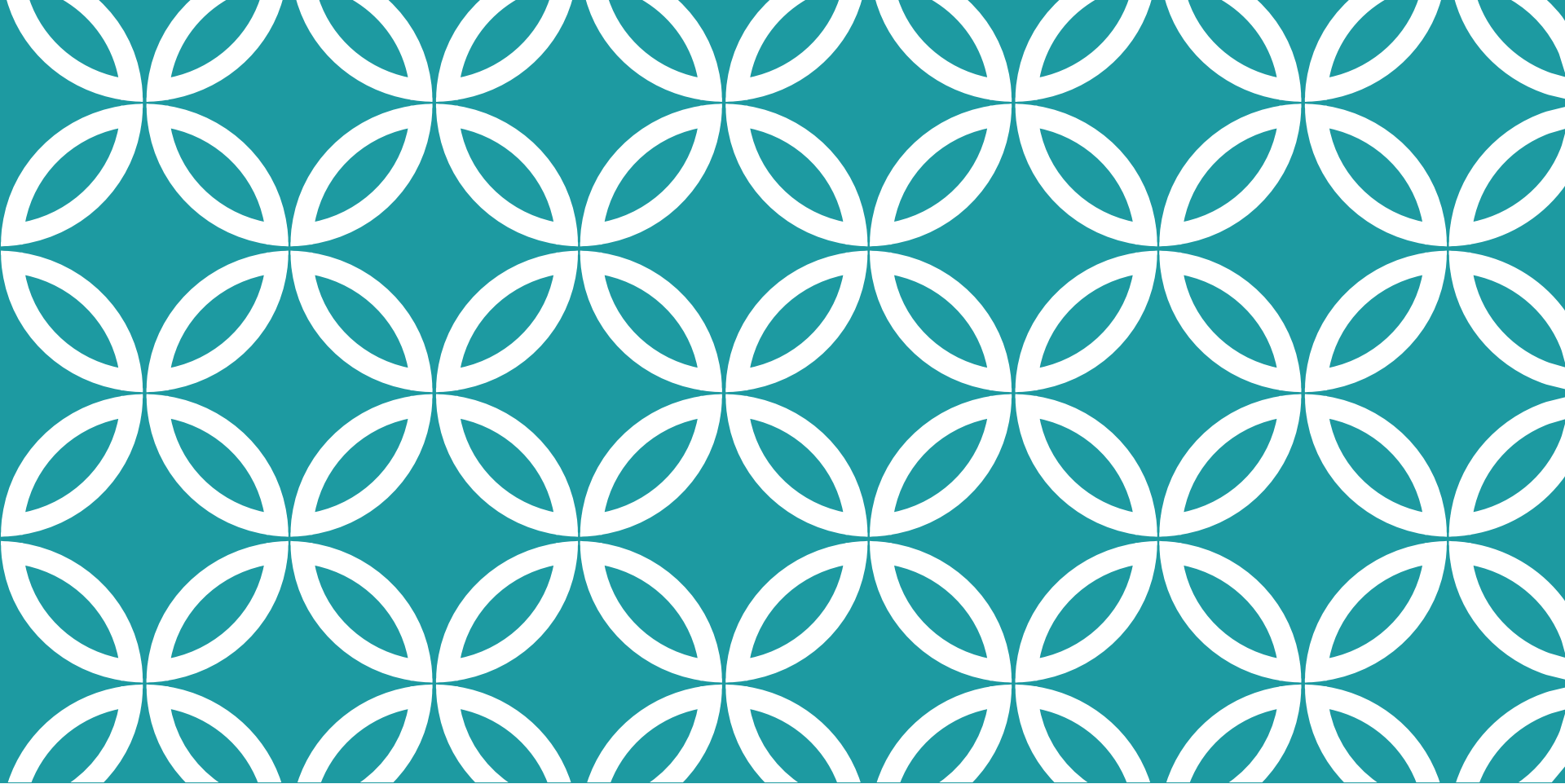
Action: Fly(p, from, to)

Precondition: $At(p, from) \wedge Plane(p) \wedge airport(from) \wedge airport(to)$

Effect: $\neg At(p, from) \wedge At(p, to)$

An action can be applied at a state iff all preconditions are true.

A solution for a planning problem is an action sequence that when executed in the initial state, results in a state that satisfies the goal.



AUTOMATED PLANNING TECHNIQUES

STRIPS, PDDL

STRIPS

STRIPS – *Stanford Research Institute Problem Solver*

One of the earliest formal languages for planning problems

In STRIPS, the world can be described with

- States - initial state and goal state
- Action schema - objects, actions, pre-conditions, and effects.

Add/delete Effects:

Add Lists - facts made true after the action (positive literals)

Delete Lists - facts made false after the action (negative literals)

Once the world is described, STRIPS will search all possible states, starting from initial state, executing applicable actions, until it reaches goal state.

STRIPS - EXAMPLE

Domain: Moving a block from table to stack

Initial State: On(A, Table), Clear(A), Clear(B)

Goal State: On(A, B)

Action (Operator): Move(A, Table, B)

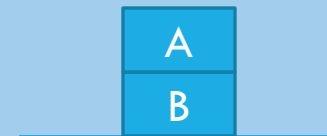
Preconditions: On(A, Table), Clear(A), Clear(B)

Add Effects: On(A, B)

Delete Effects: $\neg$ On(A, Table), $\neg$ Clear(B)



Initial State



Goal State

PDDL

PDDL - *Planning Domain Definition Language*

A standardized successor to STRIPS

PDDL extends STRIPS with richer features: typing, quantifiers, numeric fluents, temporal planning, etc.

PDDL describes four things to define a problem:

The initial state, the actions that are available in a state, the result of applying an action, and the result state

Each state is represented as **a conjunction of fluents that are ground, functionless atoms.**

Example: A state in package delivery can be represented by

$\text{At}(\text{Plane1}, \text{Bangalore}) \wedge \text{At}(\text{Plane2}, \text{Kolkata})$

PDDL

A set of ground actions can be represented using a single action schema. The schema is a lifted representation – it lifts the level of reasoning from propositional logic to a restricted subset of first order logic.

The schema consists of a name, a list of variables used in the schema, a precondition and a effect.

Example: Here is an action schema of flying a plane from “from” location to “to” location

Action(Fly(p, from, to))

Precond: $AT(p, from) \wedge Plane(p) \wedge Airport(from) \wedge Airport(to)$

Effect: $\neg AT(p, from) \wedge AT(p, to)$

Any variable is treated as existentially quantified.

The goal is a conjunction of literals (positive or negative)

EXAMPLE: AIRCARGO TRANSPORT

$AT(x, y)$ – x is at location y

$Cargo(c)$ – c is a cargo

$Plane(p)$ – p is a plane

$Airport(z)$ – location z has airport

$Action(Load(c, p, a),$

Precond: $AT(p, a) \wedge AT(c, a) \wedge Cargo(c) \wedge Plane(p) \wedge Airport(a),$

Effect: $\neg AT(c, a) \wedge IN(c, p))$

$Action(Unload(c, p, a),$

Precond: $AT(p, a) \wedge IN(c, p) \wedge Cargo(c) \wedge Plane(p) \wedge Airport(a),$

Effect: $\neg IN(c, p) \wedge AT(c, a))$

$Action(Fly(p, from, to),$

Precond: $AT(p, from) \wedge Plane(p) \wedge Airport(from) \wedge Airport(to),$

Effect: $\neg AT(p, from) \wedge AT(p, to))$

EXAMPLE: AIRCARGO TRANSPORT

$\text{Init}(\text{AT}(\text{C1}, \text{SFO}) \wedge \text{AT}(\text{C2}, \text{JFK}) \wedge \text{AT}(\text{P1}, \text{SFO}) \wedge \text{AT}(\text{P2}, \text{JFK}) \wedge \text{Cargo}(\text{C1})$
 $\wedge \text{Cargo}(\text{C2}) \wedge \text{Plane}(\text{P1}) \wedge \text{Plane}(\text{P2}) \wedge \text{Airport}(\text{SFO}) \wedge \text{Airport}(\text{JFK}))$

$\text{Goal}(\text{AT}(\text{C2}, \text{SFO}) \wedge \text{AT}(\text{C1}, \text{JFK}))$

The following plan is a solution to the problem:

[Load(C1, P1, SFO), Fly(P1, SFO, JFK), Unload(C1, P1, JFK),
Load(C2, P2, JFK), Fly(P2, JFK, SFO), Unload(C2, P2, SFO)]

EXAMPLE: THE BLOCKS WORLD

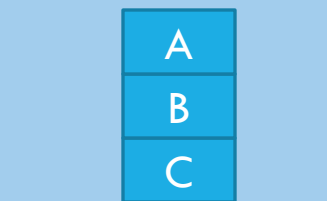
The blocks world consists of a set of cube shaped blocks sitting on a table.

A robot arm can pick up only one block at a time and move it to another position, either on the table or on top of another block.

Suppose the goal is to get block A on B and block B on C .



Initial State



Goal State

EXAMPLE: THE BLOCKS WORLD

$\text{On}(b, x)$ – Block b is on x , where x is either another block or the table.

$\text{Clear}(x)$ – nothing on top of x .

$\text{Block}(b)$ – b is a block

$\text{Move}(b, x, y)$ – Moving block b from the top of x to the top of block y . The precondition for this action is no other block be on top of b or y .

Action($\text{Move}(b, x, y)$,

PRECOND: $\text{On}(b, x) \wedge \text{Clear}(b) \wedge \text{Clear}(y) \wedge \text{Block}(b) \wedge \text{Block}(y) \wedge$
 $(x \neq y) \wedge (y \neq b) \wedge (b \neq x)$

EFFECT: $\neg \text{On}(b, x) \wedge \text{On}(b, y) \wedge \text{Clear}(x) \wedge \neg \text{Clear}(y)$)

$\text{MoveToTable}(b, x)$ – Moving block b and placing it on the table.

Action($\text{MoveToTable}(b, x)$,

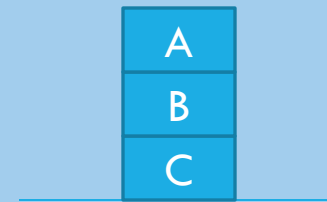
PRECOND: $\text{On}(b, x) \wedge \text{Clear}(b) \wedge \text{Block}(b) \wedge (b \neq x)$

EFFECT: $\neg \text{On}(b, x) \wedge \text{On}(b, \text{Table}) \wedge \text{Clear}(x)$)

EXAMPLE: THE BLOCKS WORLD

$\text{Init}(\text{On}(\text{A}, \text{Table}) \wedge \text{On}(\text{B}, \text{Table}) \wedge \text{On}(\text{C}, \text{A}) \wedge \text{Block}(\text{A}) \wedge \text{Block}(\text{B}) \wedge \text{Block}(\text{C}) \wedge \text{Clear}(\text{B}) \wedge \text{Clear}(\text{C}))$

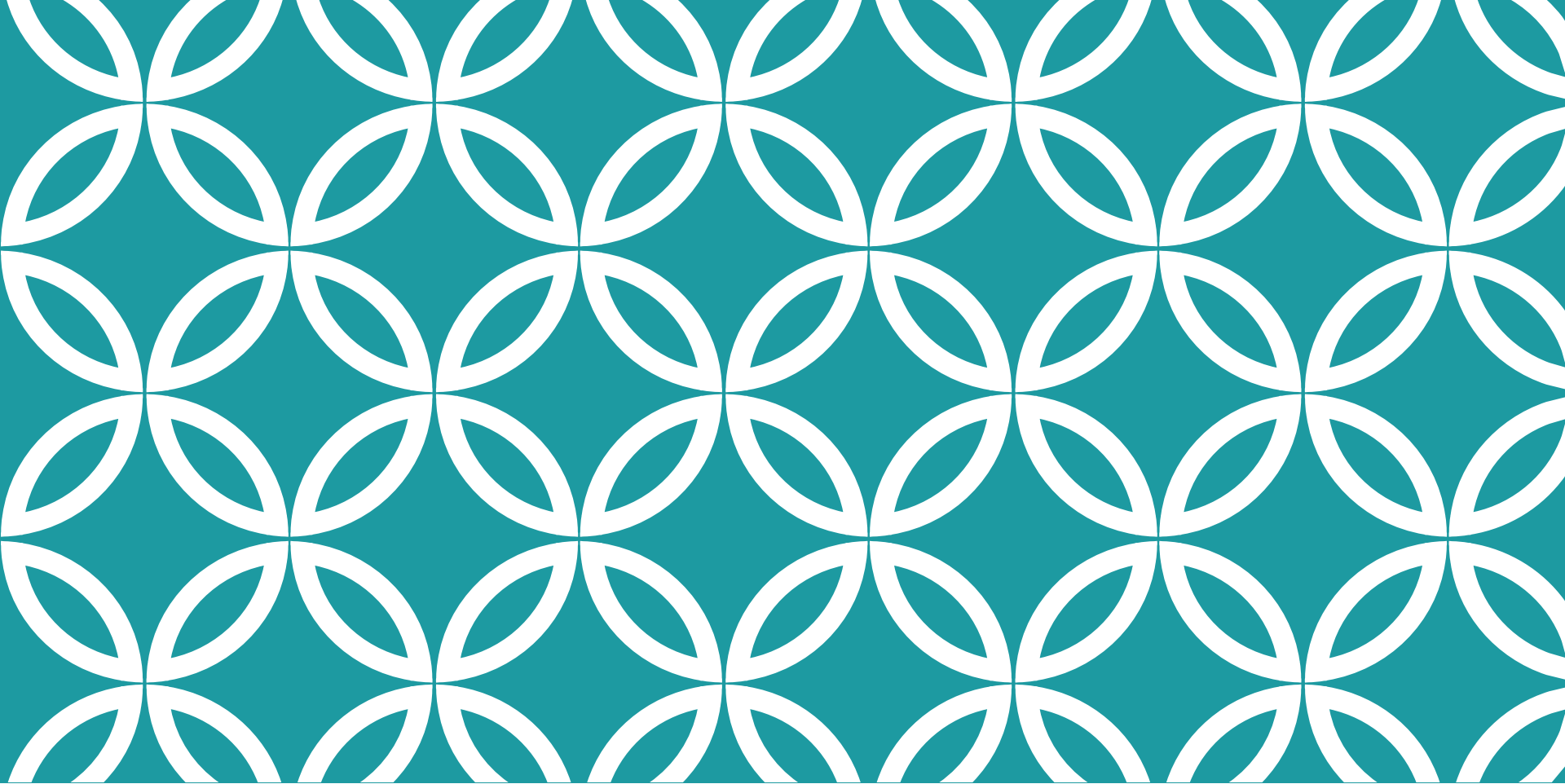
$\text{Goal}(\text{On}(\text{A}, \text{B}) \wedge \text{On}(\text{B}, \text{C}))$



We can formulate the solution as the sequence

$[\text{MoveToTable}(\text{C}, \text{A}), \text{Move}(\text{B}, \text{Table}, \text{C}), \text{Move}(\text{A}, \text{Table}, \text{B})]$

The planning of the above type are called ***totally ordered planning***. It has only strict linear sequences of actions connecting the start state to the goal state. We can't decompose the problem into sub-problems.



STATE-SPACE SEARCH FOR PLANNING

Forward(Progression)
Backward(Regression)

PLANNING AS STATE-SPACE SEARCH

Searching for a plan using state-space search

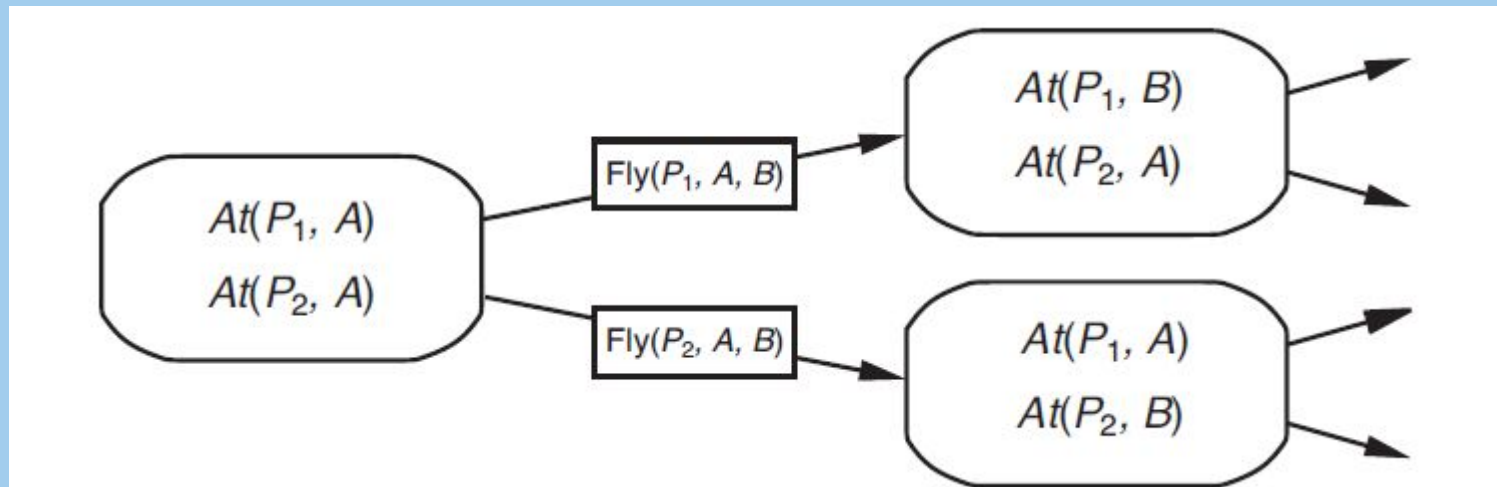
Forward /Progression Search

Backward/Regression Search

Heuristic Planning - Neither forward nor backward search is efficient without a good heuristic function.

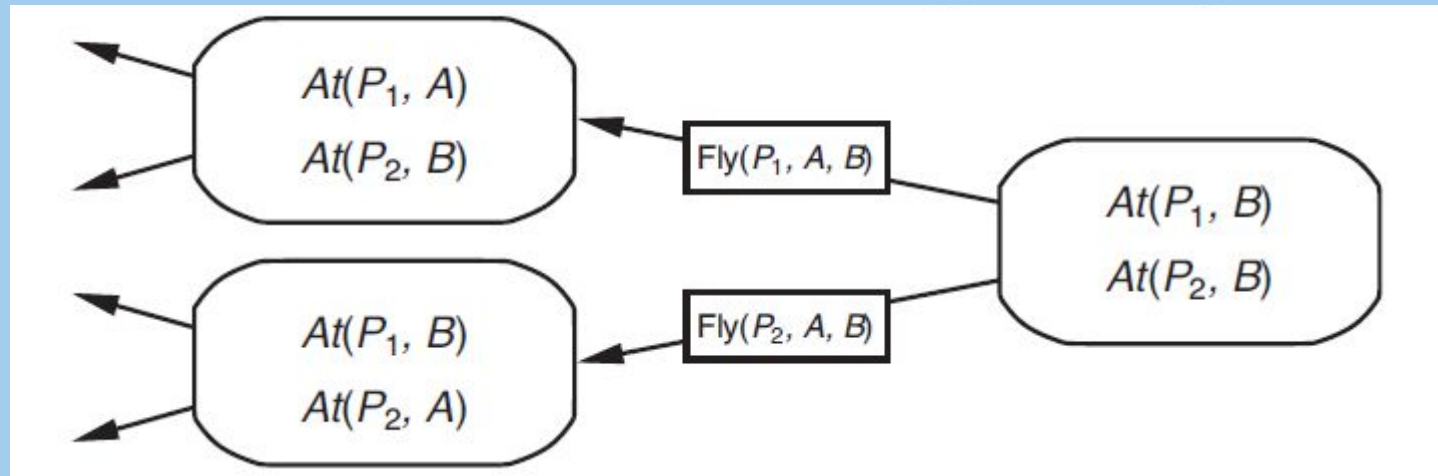
PLANNING AS STATE-SPACE SEARCH

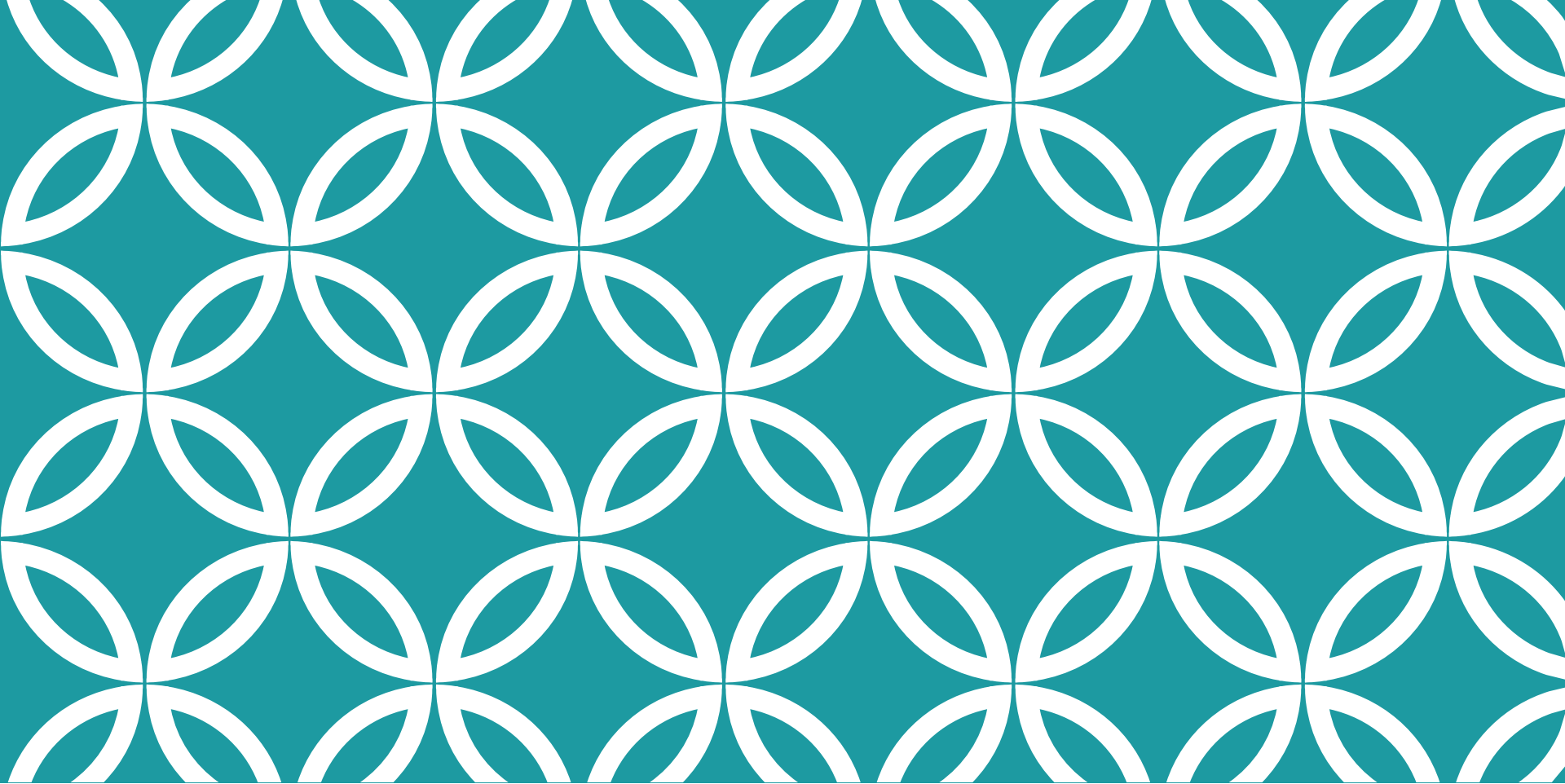
Forward (progression) search through the space of states, starting in the initial state and using the problem's actions to search forward for a member of the set of goal states.



PLANNING AS STATE-SPACE SEARCH

Backward (regression) search through sets of relevant states, starting at the set of states representing the goal and using the inverse of the actions to search backward for the initial state.





PARTIALLY ORDERED PLANNING

PARTIALLY ORDERED PLANNING

In total-order planning search, forward/Backward state-space searches are used.

- Explore only strictly linear sequences of actions, directly connected to the start or the goal
- Cannot take advantages of problem decomposition.

Works on several subgoals independently, combining subplans

Least commitment strategy:

- delaying a choice during search.
- Leave actions unordered, unless they must be sequential.

PARTIALLY ORDERED PLANNING

Consider the simple problem **of putting on a pair of shoes.**

Goal(RightShoeOn $\wedge$ LeftshoeOn) ;

Init();

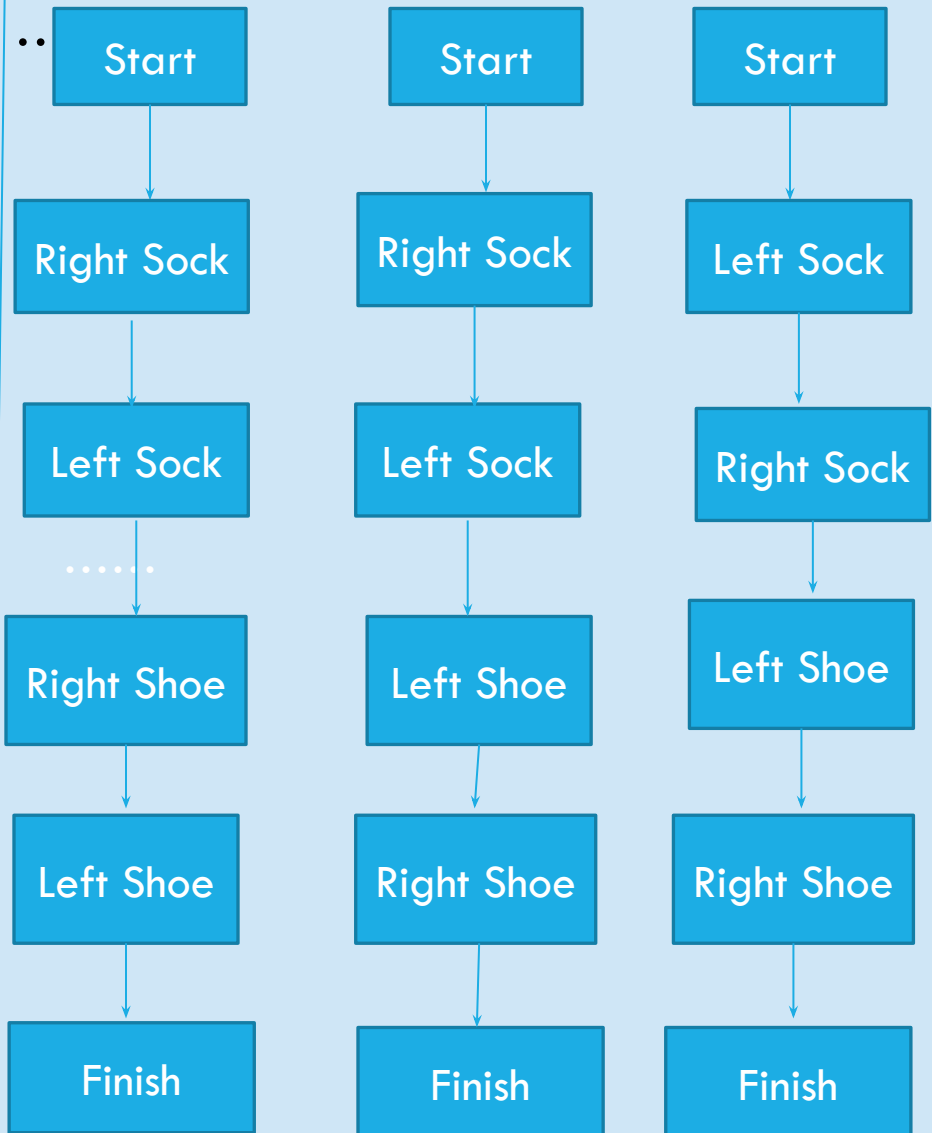
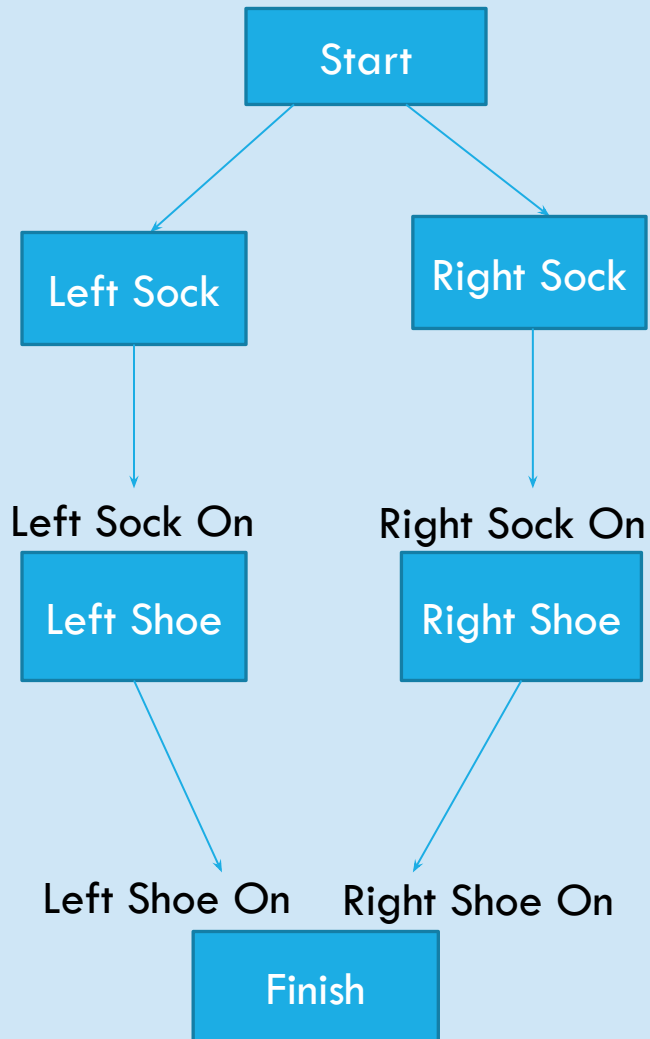
Action(RightShoe, PRECOND: RightSockOn, EFFECT: RightShoeOn)

Action(RightSock, PRECOND: , EFFECT: RightSockOn)

Action(LeftShoe, PRECOND: LeftSockOn, EFFECT: LeftShoeOn)

Action(LeftSock, PRECOND: , EFFECT: LeftSockOn)

Partial order planTotal order plans



PARTIALLY ORDERED PLANNING

A planner can plan 2 action sequences :

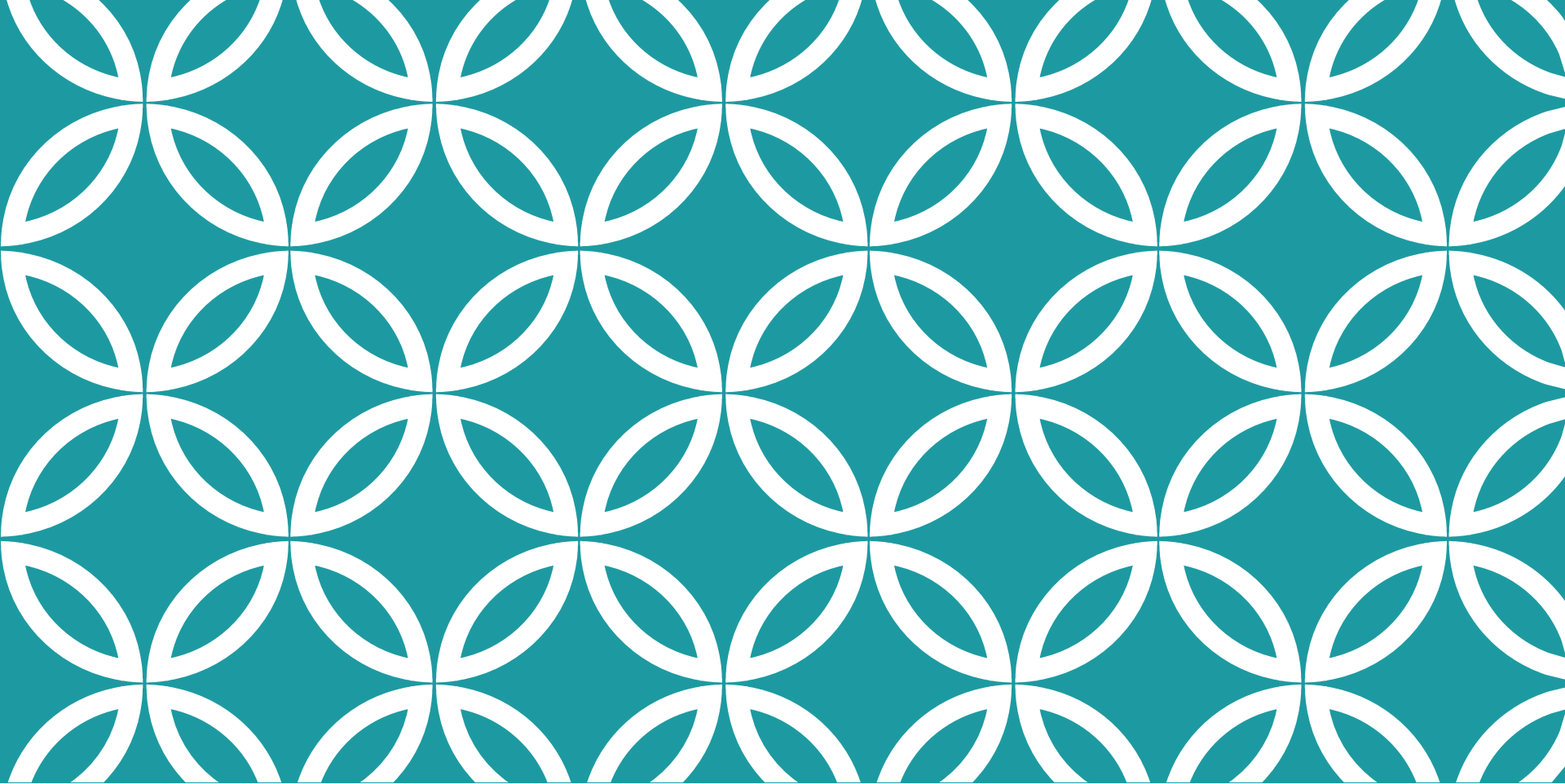
[Rightsock, RightShoe] and [LeftSock, LeftShoe]

Then the two sequences can be combined to yield the final plan. In this example, the partial order solution corresponds to six possible total order plans;

[Rightsock, RightShoe, LeftSock, LeftShoe], [Rightsock, LeftSock, LeftShoe, RightShoe], [Rightsock, LeftSock, RightShoe, LeftShoe], [LeftSock, LeftShoe, Rightsock, RightShoe], [LeftSock, Rightsock, RightShoe, LeftShoe], [LeftSock, Rightsock, RightShoe, LeftShoe]

Each of these is called **a linearization of the partial order plan**.

Any planning algorithm that can place two actions into a plan without specifying which comes first is **called a partial order planning**.



PLANNING GRAPH



PLANNING GRAPH

An Algorithm that takes a planning problem as input and generates a sequence of actions to reach the goal state using graph.

The relation between action and states are preserved. A layered graph with alternate layers of actions and effects.

The graph is constructed left to right,

- starting with the initial state,
- possible actions layer-wise,
- inserting corresponding propositions,
- until the goal proposition is reached in the propositional layer
- and these are not mutually exclusive.

Each level consists of Literals (that become/are true at a time step depending upon the action executed previously) and Actions (that have their all preconditions satisfied at the time step)

PLANNING GRAPH

Considering the problem of having cake and eating cake too.

Init (Have(Cake))

Goal ($\neg$ Have(Cake) $\wedge$ Eaten(Cake))

Action(Eat(Cake))

PRECOND: Have(Cake)

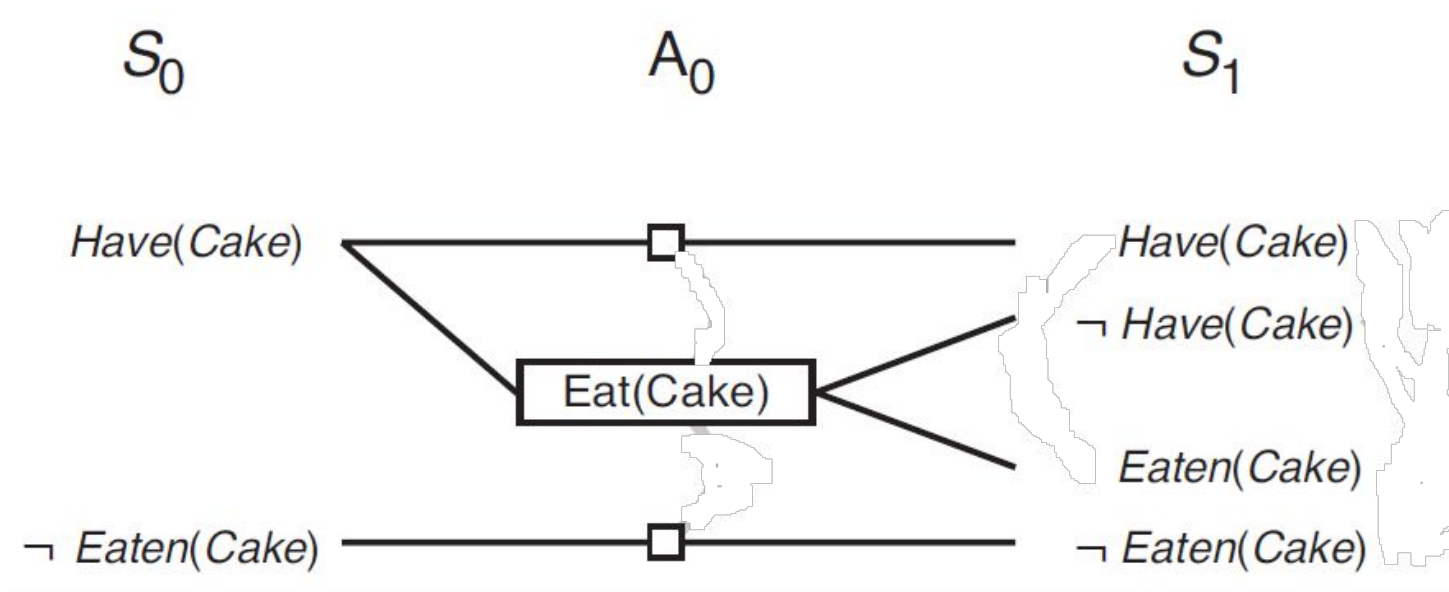
EFFECT: $\neg$ Have(Cake) $\wedge$ Eaten(Cake))

Action(Bake(Cake))

PRECOND: $\neg$ Have(Cake)

EFFECT: Have(Cake))

PLANNING GRAPH

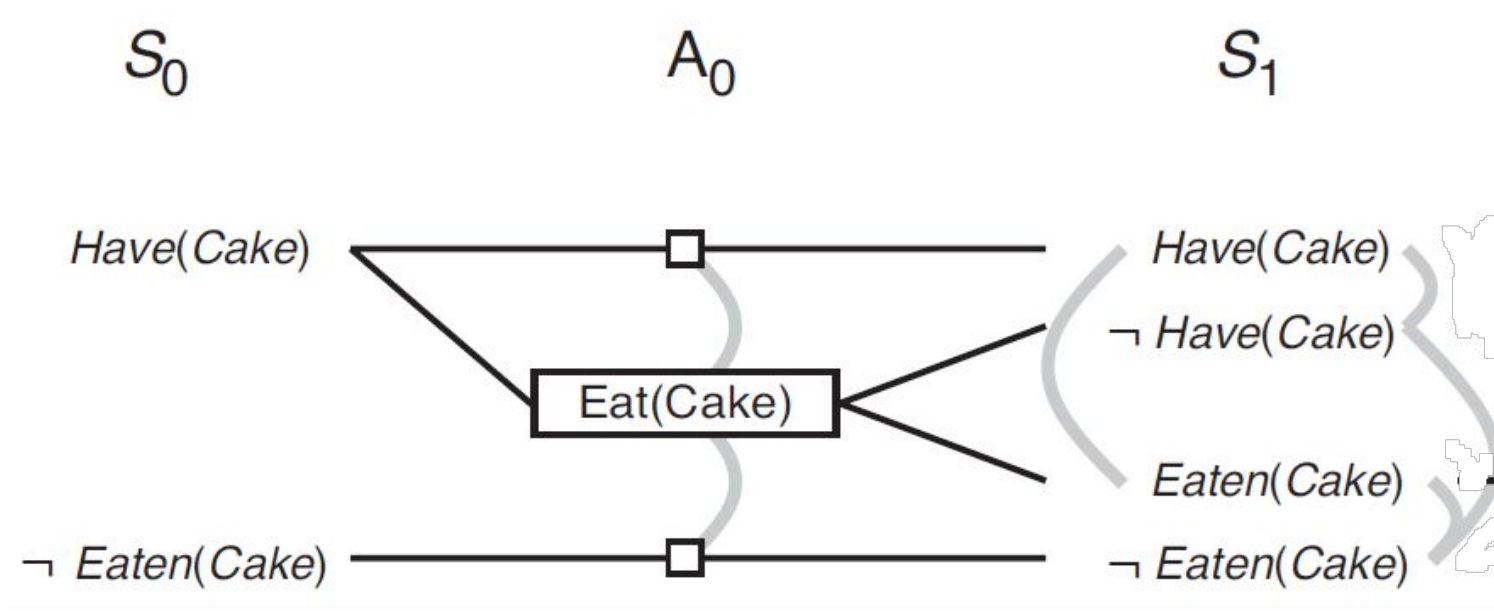


Each action at level A_i is connected to its preconditions at S_i and its effects at S_{i+1} .

A literal can persist if no action negates it. This is represented by a **persistence action** (sometimes called a *no-op*).

The figure shows one “real” action, $Eat(Cake)$, along with two persistence actions drawn as small square boxes.

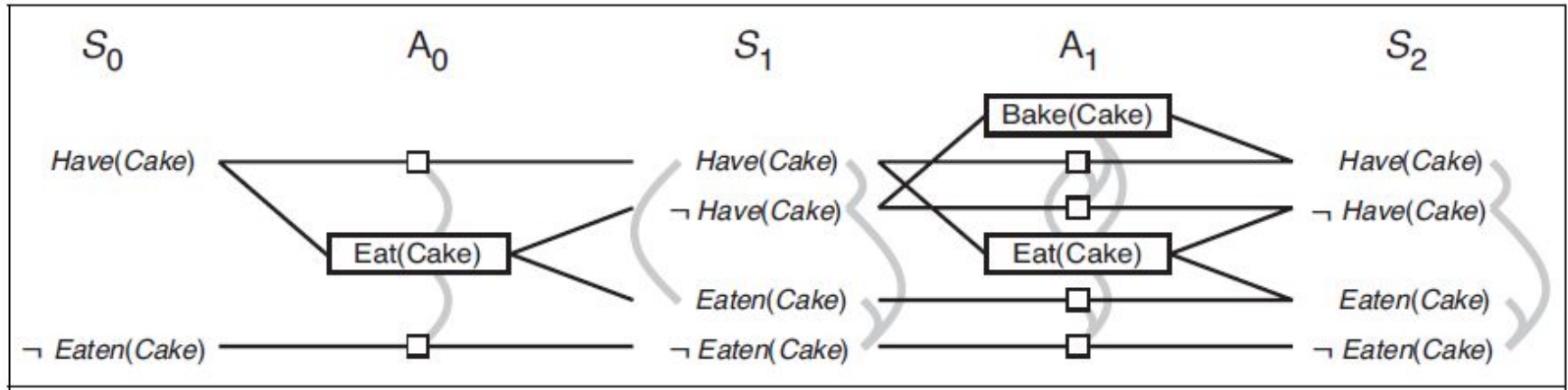
PLANNING GRAPH



Mutually Exclusive Literals and Actions:

Identify the literals and actions that are mutually exclusive based on potential conflicts. Example: $(Have(Cake), \neg Have(Cake))$, $(Eat(Cake), \neg Eat(Cake))$, $(Eaten(Cake), Have(Cake))$

PLANNING GRAPH



The planning graph for the “have cake and eat cake too” problem up to level S_2 .

Rectangles indicate actions (small squares indicate persistence actions), and straight lines indicate preconditions and effects.

Mutex links are shown as curved gray lines. Not all mutex links are shown, because the graph would be too cluttered.

In general, if two literals are mutex at S_i , then the persistence actions for those literals will be mutex at A_i and we need not draw that mutex link.

PLANNING GRAPH

A **mutex relation** holds between two actions at a given level if any of the following three conditions holds:

- **Inconsistent effects:** one action negates an effect of the other.

For example, Eat(Cake) and the persistence of Have(Cake) have inconsistent effects because they disagree on the effect Have(Cake).

- **Interference:** one of the effects of one action is the negation of a precondition of the other.

For example, Eat(Cake) interferes with the persistence of Have(Cake) by negating its precondition.

- **Competing needs:** one of the preconditions of one action is mutually exclusive with a precondition of the other.

For example, Bake(Cake) and Eat (Cake) are mutex because they compete on the value of the Have(Cake) precondition.